Blockstore 블록 저장소

개요

Blockstore는 IPFS의 핵심 저장소 추상화 계층으로, 블록 데이터를 저장하고 검색하는 인터페이스를 제공합니다. 데이터스토어 위에 얇은 래퍼를 제공하여 블록 객체를 가져오고 저장하는 깔끔한 인터페이스를 제공합니다.

Blockstore는 다양한 캐싱 전략과 최적화 기법을 적용할 수 있는 유연한 아키텍처를 가지고 있으며, 메모리 기반부터 디스크 기반까지 다양한 저장소 백엔드를 지원합니다.

블록 저장소 개념과 인터페이스

기본 Blockstore 인터페이스

```
type Blockstore interface {

// 블록 삭제

DeleteBlock(context.Context, cid.Cid) error

// 블록 존재 여부 확인

Has(context.Context, cid.Cid) (bool, error)

// 블록 가져오기

Get(context.Context, cid.Cid) (blocks.Block, error)

// 블록 크기 가져오기

GetSize(context.Context, cid.Cid) (int, error)

// 블록 저장
```

```
Put(context.Context, blocks.Block) error

// 여러 블록 일괄 저장

PutMany(context.Context, []blocks.Block) error

// 모든 키 채널로 반환

AllKeysChan(ctx context.Context) (<-chan cid.Cid, error)

}
```

기본 블록스토어 생성

```
package main


import (

"context"

"log"

"github.com/ipfs/boxo/blockstore"

"github.com/ipfs/go-datastore"

dsync  "github.com/ipfs/go-datastore/sync"

blocks  "github.com/ipfs/go-block-format"

"github.com/ipfs/go-cid"

)


func  createBasicBlockstore() blockstore.Blockstore {

// 메모리 기반 데이터스토어 생성
```

```
ds := dsync.MutexWrap(datastore.NewMapDatastore())
// 기본 블록스토어 생성
bs := blockstore.NewBlockstore(ds)
return bs
}


func basicBlockstoreExample() {
ctx := context.Background()
bs := createBasicBlockstore()
// 테스트 데이터 생성
data := []byte("Hello, IPFS Blockstore!")
block, err := blocks.NewBlockWithCid(data, cid.NewCidV1(cid.Raw, data))
if err != nil {
log.Fatal(err)
}
// 블록 저장
err = bs.Put(ctx, block)
if err != nil {
log.Fatal("블록 저장 실패:", err)
}
// 블록 존재 확인
has, err := bs.Has(ctx, block.Cid())
if err != nil {
```

```
log.Fatal("블록 존재 확인 실패:", err)
}
log.Printf("블록 존재: %v", has)
// 블록 크기 확인
size, err := bs.GetSize(ctx, block.Cid())
if err != nil {
log.Fatal("블록 크기 확인 실패:", err)
}
log.Printf("블록 크기: %d bytes", size)
// 블록 가져오기
retrievedBlock, err := bs.Get(ctx, block.Cid())
if err != nil {
log.Fatal("블록 가져오기 실패:", err)
}
log.Printf("가져온 데이터: %s", string(retrievedBlock.RawData()))
}
```

다양한 저장소 구현체 비교

1. 기본 Blockstore

```
func createStandardBlockstore() blockstore.Blockstore {
// 표준 메모리 데이터스토어
```

```
ds := dsync.MutexWrap(datastore.NewMapDatastore())
// 기본 옵션으로 블록스토어 생성
bs := blockstore.NewBlockstore(ds)
return bs
}
```

2. WriteThrough Blockstore

```
func createWriteThroughBlockstore() blockstore.Blockstore {
ds := dsync.MutexWrap(datastore.NewMapDatastore())
// WriteThrough 옵션: 블록 존재 여부를 확인하지 않고 바로 저장
bs := blockstore.NewBlockstore(ds, blockstore.WriteThrough(true))
return bs
}
```

3. NoPrefix Blockstore

```
func createNoPrefixBlockstore() blockstore.Blockstore {
ds := dsync.MutexWrap(datastore.NewMapDatastore())
// NoPrefix 옵션: "/blocks" 네임스페이스를 사용하지 않음
bs := blockstore.NewBlockstore(ds, blockstore.NoPrefix())
return bs
```

```
}
```

4. Provider Blockstore

```
import (
"github.com/libp2p/go-libp2p/core/routing"
)

func  createProviderBlockstore(provider routing.ContentProviding) blockstore.Blo
ds := dsync.MutexWrap(datastore.NewMapDatastore())
// Provider 옵션: 블록 저장 시 자동으로 네트워크에 제공자로 등록
bs := blockstore.NewBlockstore(ds, blockstore.Provider(provider))
return  bs
}
```

5. Identity Blockstore

```
func  createIdentityBlockstore(underlying blockstore.Blockstore) blockstore.Bloc
// Identity 해시를 지원하는 블록스토어
// Identity 해시는 데이터 자체가 해시인 경우 (작은 데이터용)
return  blockstore.NewIdStore(underlying)
}
```

6. Validating Blockstore

```
func  createValidatingBlockstore(underlying blockstore.Blockstore) blockstore.Bl
// 블록을 가져올 때 해시 검증을 수행하는 블록스토어
return &blockstore.ValidatingBlockstore{
Blockstore: underlying,
}
}
```

캐싱 전략과 성능 최적화

Bloom Filter 캐시

```
func  createBloomCachedBlockstore(ctx context.Context, underlying blockstore.Blo
// Bloom Filter 캐시 옵션 설정
opts := blockstore.DefaultCacheOpts()
opts.HasBloomFilterSize = 512 << 10  // 512KB Bloom Filter
opts.HasBloomFilterHashes = 7  // 7개의 해시 함수
opts.HasTwoQueueCacheSize = 0  // TwoQueue 캐시 비활성화
// 캐시된 블록스토어 생성
cachedBS, err := blockstore.CachedBlockstore(ctx, underlying, opts)
if  err != nil {
return  nil, err
```

```
}

return cachedBS, nil

}
```

TwoQueue 캐시

```
func createTwoQueueCachedBlockstore(ctx context.Context, underlying blockstore.

// TwoQueue 캐시 옵션 설정

opts := blockstore.DefaultCacheOpts()

opts.HasBloomFilterSize = 0  // Bloom Filter 비활성화

opts.HasTwoQueueCacheSize = 64 << 10  // 64K 항목 TwoQueue 캐시

// 캐시된 블록스토어 생성

cachedBS, err := blockstore.CachedBlockstore(ctx, underlying, opts)

if err != nil {

return nil, err

}

return cachedBS, nil

}
```

하이브리드 캐시 (Bloom Filter + TwoQueue)

```
func createHybridCachedBlockstore(ctx context.Context, underlying blockstore.Bl

// 하이브리드 캐시 옵션 설정
```

```
opts := blockstore.CacheOpts{
HasBloomFilterSize: 1024 << 10, // 1MB Bloom Filter
HasBloomFilterHashes: 7, // 7개의 해시 함수
HasTwoQueueCacheSize: 32 << 10, // 32K 항목 TwoQueue 캐시
}
// 하이브리드 캐시된 블록스토어 생성
cachedBS, err := blockstore.CachedBlockstore(ctx, underlying, opts)
if err != nil {
return nil, err
}
return cachedBS, nil
}
```

성능 비교 예제

```
package main

import (
"context"
"fmt"
"log"
"math/rand"
```

```
"time"

"github.com/ipfs/boxo/blockstore"

"github.com/ipfs/go-datastore"

dsync  "github.com/ipfs/go-datastore/sync"

blocks  "github.com/ipfs/go-block-format"

"github.com/ipfs/go-cid"

)



func  benchmarkBlockstore(name  string, bs blockstore.Blockstore, testBlocks []b

ctx := context.Background()

// 쓰기 성능 측정

start := time.Now()

for  _, block := range  testBlocks {

if  err := bs.Put(ctx, block);  err != nil {

log.Fatal(err)

}

}

writeTime := time.Since(start)

// 읽기 성능 측정 (랜덤 순서)

rand.Shuffle(len(testBlocks), func(i, j  int) {

testBlocks[i], testBlocks[j] = testBlocks[j], testBlocks[i]

})

start = time.Now()
```

```
for _, block := range testBlocks {
_, err := bs.Get(ctx, block.Cid())
if err != nil {
log.Fatal(err)
}
}
readTime := time.Since(start)
// Has 성능 측정
start = time.Now()
for _, block := range testBlocks {
_, err := bs.Has(ctx, block.Cid())
if err != nil {
log.Fatal(err)
}
}
hasTime := time.Since(start)
fmt.Printf("%s 성능:\n", name)
fmt.Printf(" 쓰기: %v (%v/블록)\n", writeTime, writeTime/time.Duration(len(testBl
fmt.Printf(" 읽기: %v (%v/블록)\n", readTime, readTime/time.Duration(len(testBloc
fmt.Printf(" Has: %v (%v/블록)\n", hasTime, hasTime/time.Duration(len(testBlocks)
fmt.Println()
}
```

```go
func performanceComparison() {
ctx := context.Background()
// 테스트 블록 생성
testBlocks := make([]blocks.Block, 1000)
for i := range testBlocks {
data := make([]byte, 1024) // 1KB 블록
rand.Read(data)
block, _ := blocks.NewBlockWithCid(data, cid.NewCidV1(cid.Raw, data))
testBlocks[i] = block
}
// 기본 블록스토어
ds1 := dsync.MutexWrap(datastore.NewMapDatastore())
basicBS := blockstore.NewBlockstore(ds1)
// Bloom Filter 캐시 블록스토어
ds2 := dsync.MutexWrap(datastore.NewMapDatastore())
underlyingBS := blockstore.NewBlockstore(ds2)
bloomBS, _ := createBloomCachedBlockstore(ctx, underlyingBS)
// TwoQueue 캐시 블록스토어
ds3 := dsync.MutexWrap(datastore.NewMapDatastore())
underlyingBS2 := blockstore.NewBlockstore(ds3)
twoQueueBS, _ := createTwoQueueCachedBlockstore(ctx, underlyingBS2)
// 하이브리드 캐시 블록스토어
ds4 := dsync.MutexWrap(datastore.NewMapDatastore())
```

```
underlyingBS3 := blockstore.NewBlockstore(ds4)

hybridBS, _ := createHybridCachedBlockstore(ctx, underlyingBS3)

// 성능 비교

benchmarkBlockstore("기본 Blockstore", basicBS, append([]blocks.Block{}, testBloc

benchmarkBlockstore("Bloom Filter 캐시", bloomBS, append([]blocks.Block{}, testB

benchmarkBlockstore("TwoQueue 캐시", twoQueueBS, append([]blocks.Block{}, testBl

benchmarkBlockstore("하이브리드 캐시", hybridBS, append([]blocks.Block{}, testBloc

}
```

고급 기능과 최적화

GC (Garbage Collection) 지원 블록스토어

```
func  createGCBlockstore(underlying blockstore.Blockstore) blockstore.GCBlocksto

// GC 락커 생성

gcLocker := blockstore.NewGCLocker()

// GC 지원 블록스토어 생성

gcBS := blockstore.NewGCBlockstore(underlying, gcLocker)

return  gcBS

}


func  gcExample() {

ctx := context.Background()
```

```
// GC 지원 블록스토어 생성

ds := dsync.MutexWrap(datastore.NewMapDatastore())

basicBS := blockstore.NewBlockstore(ds)

gcBS := createGCBlockstore(basicBS)

// 블록 추가 작업 (Pin Lock 사용)

pinUnlocker := gcBS.PinLock(ctx)

// 블록들 추가

testData := []byte("test data for GC")

block, _ := blocks.NewBlockWithCid(testData, cid.NewCidV1(cid.Raw, testData))

err := gcBS.Put(ctx, block)

if  err != nil {

log.Fatal(err)

}

// Pin 작업 완료

pinUnlocker.Unlock(ctx)

// GC 작업 (GC Lock 사용)

if !gcBS.GCRequested(ctx) {

gcUnlocker := gcBS.GCLock(ctx)

// GC 작업 수행

log.Println("GC 작업 시작")

// 실제 GC 로직 구현

time.Sleep(100 * time.Millisecond) // 시뮬레이션

log.Println("GC 작업 완료")
```

```
gcUnlocker.Unlock(ctx)

}

}
```

Viewer 인터페이스 활용

```
func viewerExample(bs blockstore.Blockstore) {

ctx := context.Background()

// Viewer 인터페이스 지원 확인

if viewer, ok := bs.(blockstore.Viewer); ok {

// 테스트 블록 생성

data := []byte("Hello, Viewer!")

block, _ := blocks.NewBlockWithCid(data, cid.NewCidV1(cid.Raw, data))

// 블록 저장

bs.Put(ctx, block)

// View를 통한 제로카피 접근

err := viewer.View(ctx, block.Cid(), func(data []byte) error {

// 데이터를 복사하지 않고 직접 접근

log.Printf("Viewer로 접근한 데이터: %s", string(data))

// 주의: 이 함수 내에서만 data를 사용해야 함

// data를 함수 외부로 전달하거나 저장하면 안됨

return nil

})
```

```
if err != nil {
log.Fatal("Viewer 접근 실패:", err)
}
} else {
log.Println("이 블록스토어는 Viewer 인터페이스를 지원하지 않습니다")
}
}
```

배치 작업 최적화

```
func batchOperationsExample() {
ctx := context.Background()
// 블록스토어 생성
ds := dsync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
// 대량의 블록 생성
blocks := make([]blocks.Block, 1000)
for i := range blocks {
data := fmt.Sprintf("Block data %d", i)
block, _ := blocks.NewBlockWithCid([]byte(data), cid.NewCidV1(cid.Raw, []byte(da
blocks[i] = block
}
// 개별 Put vs PutMany 성능 비교
```

```
// 개별 Put

start := time.Now()

for _, block := range blocks {

bs.Put(ctx, block)

}

individualTime := time.Since(start)

// 새로운 블록스토어로 PutMany 테스트

ds2 := dsync.MutexWrap(datastore.NewMapDatastore())

bs2 := blockstore.NewBlockstore(ds2)

start = time.Now()

bs2.PutMany(ctx, blocks)

batchTime := time.Since(start)

log.Printf("개별 Put: %v", individualTime)

log.Printf("배치 PutMany: %v", batchTime)

log.Printf("성능 향상: %.2fx", float64(individualTime)/float64(batchTime))

}
```

실제 예제: 계층화된 블록스토어

```
package main


import (

"context"
```

```
"fmt"
"log"
"time"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/go-datastore"
dsync  "github.com/ipfs/go-datastore/sync"
blocks  "github.com/ipfs/go-block-format"
"github.com/ipfs/go-cid"
)

// LayeredBlockstore는 여러 계층의 블록스토어를 관리합니다
type LayeredBlockstore struct {
// L1: 메모리 캐시 (가장 빠름)
l1Cache blockstore.Blockstore
// L2: SSD 캐시 (중간 속도)
l2Cache blockstore.Blockstore
// L3: 장기 저장소 (가장 느림, 가장 큰 용량)
l3Storage blockstore.Blockstore
// 통계
l1Hits, l2Hits, l3Hits  int64
totalRequests  int64
}
```

```go
func NewLayeredBlockstore() *LayeredBlockstore {
// L1: 메모리 캐시 (제한된 크기)
l1DS := dsync.MutexWrap(datastore.NewMapDatastore())
l1BS := blockstore.NewBlockstore(l1DS)
// L2: SSD 캐시 시뮬레이션 (실제로는 디스크 기반 데이터스토어 사용)
l2DS := dsync.MutexWrap(datastore.NewMapDatastore())
l2BS := blockstore.NewBlockstore(l2DS)
// L3: 장기 저장소 시뮬레이션
l3DS := dsync.MutexWrap(datastore.NewMapDatastore())
l3BS := blockstore.NewBlockstore(l3DS)
return &LayeredBlockstore{
l1Cache: l1BS,
l2Cache: l2BS,
l3Storage: l3BS,
}
}


func (lbs *LayeredBlockstore) Get(ctx context.Context, c cid.Cid) (blocks.Block,
lbs.totalRequests++
// L1 캐시에서 먼저 확인
if block, err := lbs.l1Cache.Get(ctx, c); err == nil {
lbs.l1Hits++
return block, nil
```

```
}
// L2 캐시에서 확인
if block, err := lbs.l2Cache.Get(ctx, c); err == nil {
lbs.l2Hits++
// L1으로 승격
lbs.l1Cache.Put(ctx, block)
return block, nil
}
// L3 저장소에서 확인
if block, err := lbs.l3Storage.Get(ctx, c); err == nil {
lbs.l3Hits++
// L2와 L1으로 승격
lbs.l2Cache.Put(ctx, block)
lbs.l1Cache.Put(ctx, block)
return block, nil
}
return nil, fmt.Errorf("블록을 찾을 수 없음: %s", c)
}


func (lbs *LayeredBlockstore) Put(ctx context.Context, block blocks.Block) error
// 모든 계층에 저장
if err := lbs.l3Storage.Put(ctx, block); err != nil {
return err
```

```
}
if err := lbs.l2Cache.Put(ctx, block); err != nil {
return err
}
return lbs.l1Cache.Put(ctx, block)
}


func (lbs *LayeredBlockstore) Has(ctx context.Context, c cid.Cid) (bool, error)
// 가장 빠른 계층부터 확인
if has, err := lbs.l1Cache.Has(ctx, c); err == nil && has {
return true, nil
}
if has, err := lbs.l2Cache.Has(ctx, c); err == nil && has {
return true, nil
}
return lbs.l3Storage.Has(ctx, c)
}


func (lbs *LayeredBlockstore) GetStats() (l1HitRate, l2HitRate, l3HitRate float
if lbs.totalRequests == 0 {
return 0, 0, 0
}
total := float64(lbs.totalRequests)
```

```
return float64(lbs.l1Hits) / total,
float64(lbs.l2Hits) / total,
float64(lbs.l3Hits) / total
}

// 나머지 인터페이스 메서드들 구현...
func (lbs *LayeredBlockstore) DeleteBlock(ctx context.Context, c cid.Cid) error
// 모든 계층에서 삭제
lbs.l1Cache.DeleteBlock(ctx, c)
lbs.l2Cache.DeleteBlock(ctx, c)
return lbs.l3Storage.DeleteBlock(ctx, c)
}

func (lbs *LayeredBlockstore) GetSize(ctx context.Context, c cid.Cid) (int, erro
// Get과 동일한 계층 순서로 확인
if size, err := lbs.l1Cache.GetSize(ctx, c); err == nil {
return size, nil
}
if size, err := lbs.l2Cache.GetSize(ctx, c); err == nil {
return size, nil
}
return lbs.l3Storage.GetSize(ctx, c)
}
```

```go
func (lbs *LayeredBlockstore) PutMany(ctx context.Context, blocks []blocks.Block

// 모든 계층에 배치 저장

if err := lbs.l3Storage.PutMany(ctx, blocks); err != nil {

return err

}

if err := lbs.l2Cache.PutMany(ctx, blocks); err != nil {

return err

}

return lbs.l1Cache.PutMany(ctx, blocks)

}


func (lbs *LayeredBlockstore) AllKeysChan(ctx context.Context) (<-chan cid.Cid,

// L3에서 모든 키를 반환 (가장 완전한 데이터)

return lbs.l3Storage.AllKeysChan(ctx)

}


func layeredBlockstoreExample() {

ctx := context.Background()

lbs := NewLayeredBlockstore()

// 테스트 블록들 생성

testBlocks := make([]blocks.Block, 100)

for i := range testBlocks {
```

```
data := fmt.Sprintf("Test block %d", i)
block, _ := blocks.NewBlockWithCid([]byte(data), cid.NewCidV1(cid.Raw, []byte(da
testBlocks[i] = block
}
// 블록들 저장
log.Println("블록들 저장 중...")
for _, block := range testBlocks {
if err := lbs.Put(ctx, block); err != nil {
log.Fatal(err)
}
}
// 랜덤 접근 패턴으로 성능 테스트
log.Println("랜덤 접근 테스트 중...")
for i := 0; i < 1000; i++ {
randomIndex := i % len(testBlocks)
block := testBlocks[randomIndex]
_, err := lbs.Get(ctx, block.Cid())
if err != nil {
log.Fatal(err)
}
}
// 통계 출력
l1Rate, l2Rate, l3Rate := lbs.GetStats()
```

```
log.Printf("캐시 히트율:")
log.Printf(" L1 (메모리): %.2f%%", l1Rate*100)
log.Printf(" L2 (SSD): %.2f%%", l2Rate*100)
log.Printf(" L3 (HDD): %.2f%%", l3Rate*100)
}

func main() {
log.Println("=== 기본 블록스토어 예제 ===")
basicBlockstoreExample()
log.Println("\n=== 성능 비교 ===")
performanceComparison()
log.Println("\n=== GC 예제 ===")
gcExample()
log.Println("\n=== 배치 작업 예제 ===")
batchOperationsExample()
log.Println("\n=== 계층화된 블록스토어 예제 ===")
layeredBlockstoreExample()
}
```

모범 사례

1. 적절한 캐싱 전략 선택

- **Bloom Filter**: 대용량 데이터셋에서 "존재하지 않음"을 빠르게 확인
- **TwoQueue**: 최근 사용된 블록과 자주 사용되는 블록을 효율적으로 캐시
- **하이브리드**: 두 방식의 장점을 결합하여 최적의 성능 달성

2. 메모리 사용량 관리

- 캐시 크기를 시스템 메모리에 맞게 조정
- 대용량 블록 처리 시 스트리밍 방식 고려
- 정기적인 메모리 사용량 모니터링

3. 배치 작업 활용

- 여러 블록을 동시에 처리할 때는 PutMany 사용
- 트랜잭션 기능이 있는 데이터스토어에서 배치 효과 극대화

4. GC 고려사항

- Pin/GC 락을 적절히 사용하여 데이터 무결성 보장
- GC 작업은 시스템 부하가 낮을 때 수행
- 중요한 데이터는 Pin으로 보호

문제 해결

일반적인 오류와 해결책

1. **메모리 부족 오류**

- 캐시 크기 줄이기
- 스트리밍 방식으로 대용량 데이터 처리
- 가비지 컬렉션 최적화

2. **성능 저하**

- 적절한 캐싱 전략 적용
- 배치 작업 활용
- 인덱싱 최적화

3. **데이터 무결성 문제**

- ValidatingBlockstore 사용
- 정기적인 데이터 검증
- 백업 및 복구 전략 수립

4. **동시성 문제**

- 적절한 락 사용
- 데이터스토어의 동시성 지원 확인
- 트랜잭션 기능 활용

5. **디스크 공간 부족**

- 정기적인 GC 수행
- 오래된 데이터 아카이빙
- 압축 기능 활용